

```

/*Find the product of digits of a number
54 = 20
92 = 18
123 = 6
*/

void main()
{
    int number = Integer.parseInt(IO.readLine("Enter a number :"));
    IO.println("Product of digits :"+getProductOfDigits(number));
}

int getProductOfDigits(int num) //0
{
    int product = 1; // = 20

    while(num!=0)
    {
        int digit = num % 10;    //digit = 5
        product = product * digit;
        num = num /10;
    }

    return product;
}

/* Find the largest digit in a number
187 = 8
245 = 5
981 = 9
*/

void main()
{
    int number = Integer.parseInt(IO.readLine("Enter a number :"));
    IO.println("The largest digit in "+number+" is :"+getMaxDigit(number));
}

int getMaxDigit(int num) //0
{
    int max = 0; //9

    while(num !=0)
    {
        int digit = num % 10;    //digit = 1

        if(digit > max)    // 1 > 9
        {
            max = digit;
        }
        num = num /10;
    }

    return max;
}

/*Find the smallest digit in a number
12 = 1
54 = 4
102 = 0
*/

void main()
{
    int number = Integer.parseInt(IO.readLine("Enter a number :"));
    IO.println("Smallest digit in "+number+" is :"+getSmallestNumber(number));
}

int getSmallestNumber(int num) //1
{
    int min = 9; //1

    while(num !=0)
    {
        int digit = num % 10;    //digit = 1

        if(digit < min)
        {
            min = digit;
        }
        num = num /10;
    }
    return min;
}

/* Find the Factorial of a number.
!5 = 5 X 4 X 3 X 2 X 1 = 120
*/

void main()
{
    int num = Integer.parseInt(IO.readLine("Enter a number :"));
    IO.println("Factorial of "+num+" is :"+getFactorial(num));
}

int getFactorial(int num)
{
    int fact = 1;

    for(int i=1; i<=num; i++)
    {
        fact = fact * i;
    }
    return fact;
}

//Find the factorial of a number by using recursive

```

What is a recursive function :

* If a function is calling itself then it is called recursive function.

* In java every function/method is executed in the stack memory so in recursive method call we need a base condition to stop the execution otherwise we will get java.lang.StackOverflowError [Stack Frame memory is not sufficient]

```

void main()
{
    int num = Integer.parseInt(IO.readLine("Enter a number :"));
    IO.println("Factorial of "+num+" is :"+getFactorial(num));
}

int getFactorial(int num) //num = 5
{
    if(num==1)
    {
        return 1;
    }

    return num * getFactorial(num -1);
}

/*

5 * getFactorial(4);

5 * 4 * getFactorial(3);

5 * 4 * 3 getFactorial(2);

5 * 4 * 3 *2 getFactorial(1);

5 * 4 * 3 *2 * 1;

*/

/* Find the reverse of a number using concatenation (String return type)
23 = 32
123 = 321
100 = 001
*/

void main()
{
    int num = Integer.parseInt(IO.readLine("Enter a number :"));
    IO.println("Reverse of "+num+" is :"+getReverse(num));
}

String getReverse(int num)
{
    String reverse = "";

    while(num !=0)
    {
        int digit = num % 10;
        reverse = reverse + digit;
        num = num / 10;
    }
    return reverse;
}

/* Find the reverse of a number int as a return type
123 = 321
678 = 876
100 = 1
*/

void main()
{
    int num = Integer.parseInt(IO.readLine("Enter a number :"));
    IO.println("Reverse of "+num+" is :"+getReverse(num));
}

int getReverse(int num)
{
    int reverse = 0;

    while(num!=0)
    {
        int digit = num % 10;
        reverse = (reverse * 10) + digit;
        num = num /10;
    }
    return reverse;
}

/*
123 = 321

int rev = 0

[rev = rev * 10 + digit]

digit = 3    rev = 3 //rev = rev * 10 + digit 3
digit = 2    rev = 32 //rev = rev * 10 + digit 32
digit = 1    rev = 321 //rev = rev * 10 + digit 321

*/

/*Check a number is Palindrome or not.
A number is a palindrome If it reads same forward and backward.
Example : 121 <-> 121
*/

/*
Logic to Check a Palindrome Number:
-----
1) Store the original number in a temporary variable.
2) Initialize reverse = 0
3) Reverse the number:
    a) Extract the last digit using num % 10, till num!=0
    b) Add it to the reversed number: reverse = (reverse * 10) + digit.
    c) Remove the last digit from the original number: num = num / 10.
4) Compare the reversed number with the temp variable.
    If both are equal, the number is a palindrome; otherwise, it is not.
*/

void main()
{
    int number = Integer.parseInt(IO.readLine("Enter a number :"));
    IO.println("Is "+number+" palindrome number :"+isPalindrome(number));
}

boolean isPalindrome(int num) //num = 121 temp =121 num = 0 rev = 121
{
    int temp = num;
    int reverse = 0;

    while(num !=0)
    {
        int digit = num % 10;
        reverse = (reverse * 10) + digit;
        num = num /10;
    }

    return reverse==temp;
}

/* Check a number is Strong number or not.

[If the sum of factorials of all individual digits of a number is equal to the number itself, then it is called a Strong Number.]

Example 145
!1 + !4 + !5 = 145

Example :
40585
!4 + !0 + !5 + !8 + !5

*/

/*
Strong number logic :
-----
1) Store the original number in a temporary variable.
2) Initialize sum = 0;
3) Extract last digit using % 10 till num !=0
4) Find the factorial of the digit. [getFactorial()]
5) Add the factorial to a sum.
6) Remove the last digit using / 10.
After processing all digits, compare the sum with the temporary number.
If equal, it's a Strong Number.
*/

void main()
{
    int number = Integer.parseInt(IO.readLine("Enter the number :"));
    IO.println("Is "+number+" Strong number :"+isStrong(number));
}

boolean isStrong(int num) //num = 145
{
    int temp = num;
    int sum = 0;

    while(num !=0)
    {
        int digit = num % 10;
        sum = sum + getFactorial(digit);
        num = num /10;
    }
    return sum==temp;
}

int getFactorial(int num)
{
    int fact = 1;
    for(int i=1; i<=num; i++)
    {
        fact = fact * i;
    }
    return fact;
}

```